

Introduction to LangChain

LangChain is an open-source framework designed to simplify the development of applications powered by large language models (LLMs). It was created to help developers build complex AI applications such as chatbots, question-answering systems, retrieval-augmented generation (RAG) pipelines, and autonomous agents in a modular and efficient way.

The core idea behind LangChain is composability. Instead of writing large amounts of custom code to connect language models with external data sources, memory systems, and tools, developers can use ready-made components that can be chained together. This approach reduces development time and makes applications easier to maintain and extend.

LangChain supports both Python and JavaScript. The Python version is the most widely used and has the richest ecosystem of integrations. It works with major LLM providers including OpenAI, Anthropic, Google, Groq, Hugging Face, and many others.

Over time, LangChain has evolved significantly. Early versions focused on simple chains and prompts. Later versions introduced agents, memory modules, document loaders, vector stores, and sophisticated retrieval techniques. The framework continues to be actively developed and is one of the most popular tools in the generative AI ecosystem.

This document provides a practical introduction to LangChain. It covers the main concepts, common components, and basic usage patterns that every developer should understand when starting to build LLM-powered applications.

Core Concepts in LangChain

LangChain is built around several fundamental building blocks. Understanding these concepts is essential before writing any code.

1. Models

Models are the interfaces to large language models. LangChain provides wrappers for both LLMs (completion models) and Chat Models (instruction-tuned models). Chat models are preferred for most modern applications because they handle multi-turn conversations more naturally and support system messages, user messages, and assistant messages.

2. Prompts

Prompts are the instructions given to the model. LangChain offers `PromptTemplate` and `ChatPromptTemplate` classes that allow developers to create reusable, parameterized prompts. This makes it easy to insert variables such as user questions or retrieved documents into a fixed template.

3. Chains

A chain is a sequence of calls to models or other components. The simplest chain takes a prompt, formats it, sends it to an LLM, and returns the result. More complex chains can include multiple steps such as retrieval, summarization, and final answer generation.

4. Memory

Memory allows applications to remember previous interactions. LangChain provides several memory types including `ConversationBufferMemory`, `ConversationSummaryMemory`, and `ConversationBufferWindowMemory`. Memory is especially important when building chatbots that need context across multiple turns.

5. Agents

Agents use a language model as a reasoning engine to decide which tools to call and in what order. Instead of following a fixed sequence of steps, an agent can dynamically choose actions based on the current situation. This makes agents suitable for open-ended tasks.

Document Loaders and Text Splitters

Most real-world applications need to work with external data. LangChain provides a wide range of document loaders that can read content from different sources.

Common document loaders include TextLoader for plain text files, PyPDFLoader for PDF documents, WebBaseLoader for websites, CSVLoader for tabular data, and DirectoryLoader for loading multiple files from a folder. Each loader returns a list of Document objects that contain both the text content and metadata.

Once documents are loaded, they are usually too long to fit into a model context window. Text splitters break long documents into smaller chunks. The most commonly used splitter is RecursiveCharacterTextSplitter. It tries to keep related text together by splitting first on paragraphs, then on sentences, and finally on characters if necessary.

Important parameters for text splitters are chunk_size and chunk_overlap. Chunk size controls how large each piece of text is. Chunk overlap ensures that important information near the boundary of two chunks is not lost. A typical starting point is a chunk size of 500 to 1000 characters with an overlap of 50 to 100 characters.

After splitting, the resulting chunks can be converted into embeddings and stored in a vector database for later retrieval.

Embeddings and Vector Stores

Embeddings convert text into numerical vectors that capture semantic meaning. Texts with similar meaning will have vectors that are close to each other in the embedding space. LangChain supports many embedding models including `OpenAIEmbeddings`, `HuggingFaceEmbeddings`, and `GoogleGenerativeAIEmbeddings`.

Once embeddings are created, they are stored in a vector store. A vector store is a specialized database optimized for similarity search. When a user asks a question, the question is also converted into an embedding, and the vector store returns the most similar document chunks.

Popular vector stores supported by LangChain include FAISS (Facebook AI Similarity Search), Chroma, Pinecone, Weaviate, and Qdrant. FAISS is frequently used for local development because it is fast and does not require an external service. Chroma is also popular for local and small-scale applications.

A typical workflow looks like this: load documents, split them into chunks, create embeddings for each chunk, store the embeddings in a vector store, and finally create a retriever from the vector store. The retriever can then be used to fetch relevant context for any user query.

This pattern forms the foundation of Retrieval-Augmented Generation (RAG), one of the most common and useful applications of LangChain.

Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation combines the strengths of retrieval systems and generative language models. Instead of asking the model to answer a question using only its internal knowledge, RAG first retrieves relevant documents from an external knowledge base and then passes those documents to the model as context.

The basic RAG pipeline consists of the following steps. First, the user question is received. Second, the question is converted into an embedding. Third, similar document chunks are retrieved from the vector store. Fourth, the retrieved chunks are inserted into a prompt along with the original question. Fifth, the language model generates an answer based on the provided context.

LangChain makes it straightforward to build RAG systems. The RetrievalQA chain is a high-level component that connects a retriever and a language model. Developers can also build custom RAG pipelines using lower-level components for greater control.

RAG is especially useful when the application needs access to private or up-to-date information that was not part of the model training data. Examples include company knowledge bases, product manuals, research papers, and internal documentation.

Good RAG systems pay careful attention to chunking strategy, embedding quality, retrieval ranking, and prompt design. Poor choices in any of these areas can lead to incomplete or incorrect answers.

Chains and the LangChain Expression Language

Early versions of LangChain used high-level chain classes such as LLMChain and SequentialChain. While these are still available, the modern recommended approach is the LangChain Expression Language (LCEL).

LCEL allows developers to compose chains using the pipe operator. A simple chain can be written as `prompt | model | output_parser`. This style is more readable and makes it easier to add streaming, asynchronous execution, and intermediate steps.

Common building blocks in LCEL include ChatPromptTemplate, Chat models, StrOutputParser, and RunnablePassthrough. These components can be combined in flexible ways to create both simple and complex workflows.

For example, a basic RAG chain in LCEL might look like this: retrieve relevant documents, format them into a prompt, send the prompt to a chat model, and parse the final string output. Because each step is a runnable, developers can inspect intermediate results and add logging or error handling easily.

LCEL also supports batching, streaming, and asynchronous calls with minimal changes to the code. This makes it suitable for both prototypes and production systems.

Agents and Tools

While chains follow a predetermined sequence of steps, agents decide what to do next based on the current state and the language model reasoning. An agent is given a set of tools and a goal. It then repeatedly chooses which tool to call until it can produce a final answer.

Tools are functions that the agent can call. Examples of tools include web search, calculator, code interpreter, database query, and custom API calls. LangChain provides many built-in tools and also makes it easy to create custom tools by wrapping ordinary Python functions.

Popular agent types include the ReAct agent, which interleaves reasoning and action steps, and more recent tool-calling agents that take advantage of native function calling support in modern language models.

Agents are powerful but require careful design. They can become expensive because they may make multiple model calls. They can also get stuck in loops or take incorrect actions if the tools and prompts are not well designed. For many applications, a well-structured chain is more reliable than an open-ended agent.

Nevertheless, agents remain one of the most interesting parts of the LangChain ecosystem and are widely used for research, data analysis, and multi-step task automation.

Memory in Conversational Applications

Stateless language model calls do not remember previous messages. For chatbots and multi-turn applications, memory is required so that the model can refer to earlier parts of the conversation.

LangChain offers several memory implementations. `ConversationBufferMemory` stores the entire conversation history. This is simple but can become expensive as the conversation grows because every previous message is sent with each new request.

`ConversationBufferWindowMemory` keeps only the last k messages. This limits cost while still providing recent context. `ConversationSummaryMemory` uses a language model to periodically summarize older parts of the conversation, keeping a compact representation of history.

More advanced memory systems can combine summary memory with entity memory or knowledge graphs. The choice of memory depends on the application requirements for cost, latency, and conversational quality.

When building production chat systems, developers often store conversation history in a database and load only the necessary context for each turn. LangChain memory classes can be integrated with external storage backends for persistence across sessions.

Common Integrations and Ecosystem

One of the strengths of LangChain is its large number of integrations. The framework does not try to implement every possible service itself. Instead it provides consistent interfaces and relies on partner packages.

Important packages include `langchain-openai`, `langchain-anthropic`, `langchain-google-genai`, `langchain-groq`, `langchain-huggingface`, and `langchain-community`. The community package contains many additional loaders, vector stores, and tools maintained by the wider community.

For observability, LangSmith is the recommended platform. It provides tracing, evaluation, and monitoring capabilities that help developers understand how their chains and agents behave in production.

LangGraph is a related library focused on building stateful, multi-actor applications with cycles and more complex control flow. Many advanced agent systems are now built with LangGraph rather than the older agent classes in LangChain.

The overall ecosystem continues to grow. New model providers, vector databases, and tools are regularly added. Developers should check the official documentation for the latest recommended packages and patterns.

Getting Started and Best Practices

To begin using LangChain, install the core packages and the specific provider packages you need. A minimal installation might include langchain, langchain-openai, and langchain-community. Always use a virtual environment and keep dependencies up to date.

Start with simple examples. Create a basic chat model call, then add a prompt template, then introduce a retriever, and only later move to agents. This incremental approach makes debugging much easier.

Prefer the LangChain Expression Language for new projects. It is more flexible and aligns with the current direction of the framework. Use high-level chains such as RetrievalQA only when you need a quick prototype.

Pay attention to cost and latency. Every model call has a price. Use smaller models for simple tasks, cache repeated queries when possible, and design retrieval so that only the most relevant context is sent to the model.

Finally, evaluate your applications. Automatic evaluation and human feedback are both important. Tools such as LangSmith make it practical to measure quality over time and detect regressions when prompts or models change.

LangChain remains one of the most practical frameworks for building real applications with large language models. By mastering its core concepts and following good engineering practices, developers can move from simple prototypes to reliable production systems.

End of Document